# MISSION ARCHITECTURALE : Infrastructure pour le futur LLM Local (Nova) dans KADERBOT

## CONTEXTE & OBJECTIF
Nous préparons KADERBOT à accueillir son propre modèle de langage local léger : **Nova** (basé sur un fine-tuning local `SmolLM2-1.7B` exporté en `.gguf` / local inference).

Étant donné que Nova est un LLM local léger (1.7B) exécuté hors-ligne sur appareil (ex: Pydroid 3 / moteur local), il a besoin d'un système de **Mémoire Chirurgicale (Prompt Contextuel Dynamique)** pour fonctionner de manière fluide sans dériver ni halluciner lors de longues conversations.

## RÈGLE STRICTE DE SEGMENTATION (ATTENTION)
1. **Fournisseurs Cloud (Gemini / Groq)** : Leurs pipelines de chat et leurs requêtes API doivent rester 100% INTACTS et STANDARD. Ils gèrent leur propre contexte et ne doivent PAS recevoir cette injection de mémoire locale.
2. **Mode Local / Moteur Nova (Pydroid 3 & Local Engine)** : C'est le SEUL endroit où cette mémoire chirurgicale s'applique.

---

## MODIFICATIONS DE CODE SOURCE

### 1. Dans `src/core/cognitiveEngine.ts`
Ajouter une fonction d'injection dédiée exclusivement au moteur local Nova :

```typescript
export function construirePromptContextuelNova(
  question: string,
  memoire: MemoryStore,
  regles: RulesStore,
  liens: LinksStore,
  synonymes: SynonymsStore,
  contexte: string[],
  systemPromptDefaut?: string
): { promptComplet: string; faitTrouve: string | null; source: string } {
  // Recherche chirurgicale du fait le plus pertinent dans la mémoire locale JSON
  const qaMatch = trouverQuestionReponse(question, memoire);
  let faitTrouve: string | null = null;
  let source = 'systeme';
  let sujetDeclencheur = '';

  if (qaMatch) {
    faitTrouve = qaMatch.entry.def;
    source = 'local_memoire';
    sujetDeclencheur = qaMatch.sujet;
  } else {
    const defCheck = estRequeteDefinition(question);
```

```
    const candX = defCheck.estDef && defCheck.concept ? defCheck.concept :
extraireX(question, memoire, regles, synonymes, contexte);
    if (candX) {
      const defLocal = phraseDef(candX, question, memoire, regles, liens, synonymes);
      if (defLocal) {
        faitTrouve = defLocal;
        source = 'local_memoire';
        sujetDeclencheur = candX;
      }
    }
  }

  // Formatage du bloc de mémoire chirurgicale pour Nova
  const blocMemoire = faitTrouve
    ? `[FAIT VÉRIFIÉ EN MÉMOIRE LOCALE SUR "${sujetDeclencheur.toUpperCase()}"]\n-
${faitTrouve}`
    : `[MÉMOIRE LOCALE]\nAucun fait spécifique pré-enregistré sur ce sujet exact. Réponds
avec fluidité, politesse et précision.`;

  const instructionsSysteme = systemPromptDefaut ||
    "Tu es KADERBOT Nova, une assistante virtuelle bilingue (FR/EN), polie, chaleureuse et
concise. " +
    "Tu réponds toujours en français, en 2 à 4 phrases maximum. " +
    "Ne parle jamais à la place de l'utilisateur et n'invente pas d'informations fausses.";

  const promptComplet = `${instructionsSysteme}\n\n${blocMemoire}\n\nUtilisateur:
${question}\nKADERBOT Nova:`;

  return { promptComplet, faitTrouve, source };
}
```

---

Dans server.ts (Générateur / Mode Embarqué Local)
Mettre à jour le template du script Python généré dans /api/export-pydroid pour qu'il intègre la fonction d'injection de prompt pour Nova :

```
def construire_prompt_contextuel_nova(question):
    """
    Construit le prompt chirurgical spécifiquement adapté pour le modèle local Nova.
    """
    qa_match = trouver_question_reponse(question, memoire)
    fait_connu = ""
    sujet_ref = ""

    if qa_match:
        fait_connu = qa_match["entry"].get("def", "")
        sujet_ref = qa_match["sujet"]
    else:
```

```
        x = extraire_x(question, memoire, regles, synonymes, contexte)
        if x and x in memoire:
            defs = [item["def"] for item in memoire[x]]
            fait_connu = " ; ".join(defs)
            sujet_ref = x

    if fait_connu:
        bloc_ctx = f"[FAIT VÉRIFIÉ EN MÉMOIRE LOCALE SUR '{sujet_ref.upper()}']\n-
{fait_connu}"
    else:
        bloc_ctx = "[MÉMOIRE LOCALE]\nAucun fait spécifique stocké. Réponds avec fluidité
et concision."

    return f"{SYSTEM_PROMPT}\n\n{bloc_ctx}\n\nUtilisateur: {question}\nKADERBOT Nova:”
```

___

VERIFICATION FINALE

Vérifier que la route d'API /api/chat gère Gemini et Groq sans modifier leur structure de messages.
Vérifier la compilation TypeScript avec npm run lint.